Python Engineering - Reference Guide

CLASS 15 - Exceptions & Debugging | Sintaxis y herramientas

Esta guia funciona como referencia extendida. Incluye sintaxis, significado, entradas/salidas mentales, casos de uso y errores habituales.

1. Que es una excepcion?

Una excepcion es un objeto que representa que, durante la ejecucion, ha ocurrido una situacion que impide continuar normalmente por ese camino.

```
float('12.5')          # OK
float('NOT_A_NUMBER')  # raises ValueError
```

La excepcion interrumpe el flujo normal y busca un manejador compatible.

2. try

Que hace: delimita el codigo cuya excepcion queremos gestionar.

```
try:
    trajectory = load_trajectory_csv(file_path)
```

No debemos meter medio programa dentro de try 'por si acaso'. Conviene proteger solo la operacion cuyo fallo sabemos interpretar.

3. except ExceptionType

Que hace: captura una excepcion compatible si se produce dentro del try.

```
except FileNotFoundError:
    print('File does not exist')
```

Si la excepcion es de otro tipo, ese except no la captura y la busqueda continua.

4. as error

Guarda el objeto excepcion en una variable local para poder inspeccionarlo.

```
except ValueError as error:
    print(type(error).__name__)
    print(error)
```

Expresion	Resultado conceptual
type(error).__name__	Nombre de la clase de excepcion
str(error) / print(error)	Mensaje asociado a la excepcion

5. else

Se ejecuta solo si el bloque try termina sin lanzar ninguna excepcion.

```
try:
    data = load()
except ValueError:
    ...
else:
    analyze(data)
```

Es util para separar 'operacion que puede fallar' de 'trabajo que solo tiene sentido si hubo exito'.

6. finally

Se ejecuta tanto si hubo excepcion como si no.

```
try:
```

```
    ...
finally:
    print('Always runs')
```

Se usa para acciones que deben ocurrir siempre. Con archivos, `with open(...)` ya gestiona el cierre, por lo que no necesitamos finally para cerrar ese archivo.

7. raise

Que hace: lanza deliberadamente una excepcion.

```
if header != expected_header:
    raise ValueError('Invalid CSV header')
```

Sirve para convertir una regla de nuestro programa en un fallo explicito. Es una forma de decir: 'continuar con estos datos produciria resultados no fiables'.

8. ValueError

Indica que el tipo general de operacion tiene sentido, pero el **valor** recibido no es valido para esa operacion.

```
float('abc')
# ValueError
```

Tambien podemos generarlo nosotros cuando un archivo tiene una cabecera/estructura que viola nuestro contrato.

9. FileNotFoundError

Aparece cuando intentamos abrir un archivo que no existe en la ruta indicada.

```
open('missing.csv', 'r')
# FileNotFoundError
```

Antes de asumir que 'Python no encuentra el archivo', revisa la ruta completa y el current working directory si usas rutas relativas.

10. SyntaxError vs exception de ejecucion

Tipo	Momento	Ejemplo
SyntaxError	Python no puede interpretar el codigo	if x > 0 (missing colon)
Runtime exception	El codigo empieza a ejecutarse y algo falla	float('abc')

Un try/except no es la herramienta normal para corregir sintaxis invalida.

11. len()

Devuelve el numero de elementos de un objeto con longitud.

```
values = ['100.0', '20.0', '0.0']
len(values)
# 3
```

En EX23 lo usamos para validar el numero de columnas.

12. enumerate()

Permite iterar obteniendo a la vez un contador y el elemento.

```
for line_number, line in enumerate(file, start=2):
    ...
```

Recibe: iterable y opcionalmente start. **Produce en cada iteracion:** (indice, elemento).

13. type(...).__name__

```
type(error).__name__
# 'ValueError'
```

Es util didacticamente para ver que las excepciones tambien son objetos pertenecientes a clases.

14. Evitar except: sin tipo

```
# Too broad for this exercise
try:
    ...
except:
    print('Something failed')
```

Captura practicamente cualquier excepcion y puede ocultar errores que no sabemos gestionar. Preferimos capturar tipos concretos que entendemos.

15. except Exception as error

Es menos amplio que un `except:` desnudo pero sigue capturando muchisimos errores comunes.

```
except Exception as error:
    ...
```

Puede ser apropiado en ciertos limites de una aplicacion, pero **no es nuestra opcion por defecto**. En EX23 queremos distinguir claramente archivo ausente de datos invalidos.

16. Exception message

El mensaje debe ayudar a actuar, no solo decir 'error'.

```
# Weak
raise ValueError('Error')

# Better
raise ValueError(
    f'Line {line_number}: expected 3 columns, got {len(values)}'
)
```

17. Traceback

Un traceback describe la cadena de llamadas que estaba activa cuando una excepcion no fue gestionada.

```
Traceback (most recent call last):
  File 'main.py', line 20, in main
    load_trajectory_csv(path)
  File 'file_io.py', line 31, in load_trajectory_csv
    v = float(values[1])
ValueError: could not convert string to float: 'NOT_A_NUMBER'
```

Lectura practica: empieza por el final para identificar tipo y mensaje; despues sube hasta la primera linea relevante de tu propio codigo.

18. Breakpoint

Un breakpoint pausa la ejecucion **antes** de una linea para inspeccionar el estado del programa.

```
Before:
v = float(values[1])

Inspect:
values
values[1]
type(values[1])
```

19. Step Over / Step Into / Continue

Comando	Idea sencilla
Continue	Seguir hasta el siguiente breakpoint
Step Over	Ejecutar la linea actual sin entrar en funciones llamadas
Step Into	Entrar dentro de la funcion que se llama

Comando	Idea sencilla
Step Out	Terminar la funcion actual y volver al caller

20. Variables y Call Stack en VS Code

Variables muestra valores en el scope actual. **Call Stack** muestra la cadena de funciones que nos ha llevado al punto actual.

```
main()
  -> test_file()
      -> load_trajectory_csv()
          -> float(...)  # failure
```

> El Call Stack es la version interactiva de una idea parecida a la que luego vemos escrita en un traceback.

21. Contrato de una funcion

Una funcion tiene expectativas sobre sus entradas y garantias sobre sus salidas. Si una entrada viola una regla esencial, una excepcion puede expresar que el contrato no se cumple.

```
load_trajectory_csv(path)

expects:
- existing readable file
- expected header
- 3 numeric columns per row

returns:
- numerical trajectory array
```

22. Cheat sheet final

```
try:
    risky_operation()
except FileNotFoundError as error:
    handle_missing_file(error)
except ValueError as error:
    handle_bad_data(error)
else:
    use_successful_result()
finally:
    always_run_this()

raise ValueError('clear message')

DEBUG:
breakpoint -> inspect variables -> predict -> step

TRACEBACK:
bottom -> exception type/message -> move upward to your code
```